Ders 7 Genişletilmiş Notu: Bağlantılı Listeler: Ekleme, Silme, Çift Yönlü ve Dairesel Listeler

Ders 7 Genişletilmiş Notu: Bağlantılı Listeler: Ekleme, Silme, Çift Yönlü ve Dairesel Listeler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Giriş: Neden Önce Diziler?
- 2. Dizilerin Kısa Tekrarı
- 3. Dizide Araya Ekleme ve Silmenin Maliyeti
- 4. Bağlantılı Liste Kavramı
- 5. Bağlantılı Liste Üzerindeki İşlemler
- 6. C'de Düğüm Tanımı ve Listeyi Gezme
- 7. Listeye Eleman Ekleme
- 8. İki Dizi ile Bağlantılı Liste ve Sıralı Listeye Ekleme
- 9. Ekleme Fonksiyonlarının C Kodu
- 10. Listeden Eleman Silme
- 11. Bağlantılı Liste ile Insertion Sort
- 12. Çift Yönlü Bağlantılı Liste
- 13. Dairesel Bağlantılı Liste ve Josephus Problemi
- 14. Dizi mi, Bağlantılı Liste mi?

1. Giriş: Neden Önce Diziler?

Hoca derse, bu haftanın ilk temel veri yapısına giriş haftası olduğunu söyleyerek başladı. Konu bağlantılı listelerdir (linked list). Ancak bağlantılı listeye geçmeden önce, onun yerini aldığı ya da ona iyi bir alternatif olduğu yapıyı, yani dizileri (array) hızlıca hatırlattı. Amaç şudur: Dizi ne zaman avantajlı, ne zaman dezavantajlıdır? Bunun yerine bağlantılı liste kullanınca ne değişir? Dersin sonunda bu karşılaştırma ayrıca yapıldı (Bölüm 14).

2. Dizilerin Kısa Tekrarı

Dizi, aynı türdeki verileri bir arada tutan ve bunlardan bir veri öbeği oluşturan temel veri yapısıdır. Dizinin en önemli özelliği, her elemanına bir indis üzerinden erişilebilmesidir. Bu erişimin maliyeti eleman sayısından bağımsızdır, yani $O(1)$'dir.

Diziler, özellikle üst seviye programlama dillerindeki pek çok karmaşık veri yapısının temelini oluşturur. Vektör ve matris gibi yapılar temelde dizidir. Çok boyutlu diziler matrisleri ya da daha yüksek boyutlu veri öbeklerini oluşturur. Liste, yığın (stack), kuyruk (queue), heap ve karakter dizileri (string) gibi yapılarda da diziler sıkça kullanılır. Hoca bunların uygulamalarının ilerleyen haftalarda görüleceğini söyledi.

2.1 İndisleme Türleri

Programlama dillerinde birbirine alternatif birkaç indisleme yaklaşımı vardır:

- **Sıfır tabanlı indisleme:** C ve benzeri dillerde kullanılır. İlk eleman 0. indiste, son eleman $n-1$. indistedir.
- **Bir tabanlı indisleme:** Pascal gibi dillerde (hocaya göre BASIC'te de) kullanılır. İlk eleman 1. indiste, son eleman n. indistedir.
- **Diğer indisleme biçimleri:** Python gibi dillerde negatif indis kullanılabilir. Negatif indis, dizinin sonundan başına doğru, tersten indisleme içindir. 0. eleman ilk gözü gösterirken -1. eleman "bir geri git", yani dönüp dolaşıp son elemana git demektir; -2 sondan ikincidir. Bazı dillerde ise sayım değerleri (enumeration) indis olabilir: örneğin kırmızı, yeşil, mavi gibi renk adları. Bazı dillerde karakterler de (A, B, C, ...) indis olarak kullanılabilir.

Hoca, C'de kodlarken doğal olarak sıfır tabanlı indisleme kullanıldığını, ama algoritmalar akış diyagramıyla ifade edilirken genellikle $1..n$ gösteriminin tercih edildiğini hatırlattı. Bu daha genel bir ifadedir. Akışta "1'den n'e" diye okunan bir döngü C'ye çevrilirken `i = 0`'dan `i < n` olana kadar, yani `i` $n-1$ olana kadar yazılır. Sınır ve son koşul kontrolleri buna göre yapılmalıdır.

2.2 C'de Dizi Kullanırken Programcının Sorumlulukları

C'de `A[i]` değerini kullanmadan önce o göze anlamlı bir değer atamak tamamen programcının sorumluluğundadır. Atama yapılmazsa, gözde şans eseri sıfır olabilir. Bellek alanı daha önce kullanılmış ve programa tahsis edilmişse, oradan kalan herhangi bir sayısal değer (mimariye göre 32 ya da 64 bitlik) bulunur.

Aynı şekilde pozitif ve dizi boyutundan küçük indis kullanmak da programcının sorumluluğundadır. C'de belleğe doğrudan erişim imkânı olduğu için, dikkatsiz kodlamada bütün bellek bir dizi gibi kullanılabilir. Program, erişimi olmaması gereken bellek alanlarını okuyup yazabilir. Hoca bunu derleyici modlarıyla engellemenin programcının elinde olduğunu, ama bu imkânın her zaman var olduğunu söyledi. Bu hem kötü niyetli hem de dikkatsiz kodlamada sorun çıkarır.

> **Hocanın vurgusu:** C'de dizi kullanırken sınır kontrollerine ve indislere ekstra dikkat edilmelidir. Aksi hâlde program başka bellek alanlarına "sarkabilir".

2.3 Statik ve Dinamik Diziler

Statik dizi: Dizinin tipi ve boyutu derleme zamanından önce tanımlanır. Derleyici bellekte bir yer ayırır ve her göze indisle erişilecek şekilde adresleri kurgular. Örneğin `int a[10];` tanımıyla bellekte bir adresten (hocanın örneğinde `0xa1ff` gibi bir adres) başlayan, 32 bitlik tamsayılar için 4'er bayt ilerleyen 10 göz açılır.

Karakter dizisinde iki yol vardır. Birincisi çift tırnak kullanmaktır. Örneğin `"dizi"` yazmak şu anlama gelir: 0. göze `'d'`, 1. göze `'i'`, 2. göze `'z'`, 3. göze `'i'` koy. Hoca bunları tek tırnakla yazdı; bunlar 8 bitlik sayılardır ama ASCII kodlamasıyla küçük harf olarak görülmek istenir. Unutulmaması gereken nokta, 4. gözde bir de null karakter (`'\0'`) bulunmasıdır. Yani 4 karakterlik bir dizi bu şekilde tanımlanınca bellekte 5'lik yer açılır. Burada boyut verilmemiştir; boyut ilk değer atamasıyla oluşur. Karakterler tek tek tırnakla verilseydi, sona `'\0'` koymak programcının sorumluluğunda olurdu.

Dinamik dizi: İhtiyaç oldukça boyutu değiştirilebilen esnek bir yapıdır. İşaretçi aritmetiğinden (pointer arithmetic) yararlanılır. Örneğin `c` adında bir tamsayı işaretçisi tanımlanır ve standart kütüphanedeki `malloc` ile 10 tamsayılık yer ayrılır. Hoca, bu yerin fiziksel olarak nerede olduğunun önemli olmadığını, ama mantıksal olarak 0'dan 9'a kadar indislerle erişilebilen bir yapı oluştuğunu söyledi. Artık `c[0]`, `c[1]`, ..., `c[9]` diye kullanılır.

3. Dizide Araya Ekleme ve Silmenin Maliyeti

Statik tanımlanmış bir diziye boyutundan fazla eleman eklenemez. Dinamik dizide eklenebilir, ama bunun için fonksiyon çağrıları ve `realloc` çağrıları yapmak gerekir.

Hocanın örneği sıralı bir diziydi: `2, 3, 6, 8, 10, ..., 24`. Bu diziye 8 ile 10 arasına 9 eklemek istensin. Yapılması gerekenler:

1. Dizinin boyutu `realloc` ile 1 artırılır.

2. 9'un gireceği konum i ise, n. konumdan $i+1$. konuma kadar bütün değerler teker teker bir sonraki indise kopyalanır.
3. 9 boşalan göze yazılır.

Bunun en kötü durumu en başa eleman eklemektir. O zaman bütün elemanlar birer göz kaydırılır. n ya da $n-1$ eleman kaydırılacağı için maliyet $O(n)$ mertebesindedir.

Silmede de durum aynıdır. Örneğin aradan 6 silinecekse, 6'nın sağındaki bütün elemanlar birer birer sola taşınır. Ardından `realloc` ile en sondaki göz boşaltılır. En kötü durumda yine $O(n)$ işlem yapılır.

Yani klasik dizi yaklaşımında araya ya da başa eleman eklemek doğrusal maliyetlidir. Hoca şu soruyu sordu: Böyle problemler varsa, bu maliyeti düşürebilir miyiz? Örneğin sürekli başa eleman ekleniyorsa, bunu sabit sayıda işlemle yapabilir miyiz? Cevap evettir; çözüm başka bir veri yapısı kullanmaktır. Bu veri yapısı bağlantılı listedir (linkli liste).

4. Bağlantılı Liste Kavramı

Bağlantılı liste, dinamik olarak yaratılan ve çalışma sırasında hem içeriği hem de sıralaması dinamik olarak değiştirilebilen bir veri yapısıdır.

Bunu sağlamak için atomik elemanlar yaratılır. Bunlara düğüm (node) denir. Hoca düğümü bir kutucukla çizdi. Kutucuğun bir kısmında veri (data) vardır. Bu bir tamsayı, karakter, string, kayan noktalı sayı, aklınıza ne gelirse olabilir. Diğer kısmında bir işaretçi (pointer) vardır. Bu işaretçi listede kendisini takip eden düğümü gösterir.

Hoca tahtada D1, D2, D3, D4 verili düğümler çizdi. Düğümler bellekte farklı yerlerde durabilir. D1'in işaretçisi bellekteki başka bir noktada duran D2'yi, D2'ninki D3'ü gösterir ve bu böyle sürer. Listenin sonuna eleman eklenmek istenirse, bellekte bir yerde yeni düğüm oluşturulur ve son düğümün işaretçisi onu gösterecek şekilde ayarlanır.

Yani tutulması gereken veri ile aynı yapının içinde bir sonraki elemanı gösteren işaretçi bir paket hâlinde düğüm olarak oluşturulur. Düğümler işaretçilerle birbirini gösterecek şekilde bağlanırsa buna bağlantılı liste denir.

Bu yapının özellikleri şunlardır:

- **Dinamik boyut:** İstenildiği kadar düğüm oluşturulup birbirine bağlanabilir. Liste 1, 5, 10 ya da n düğümden oluşabilir. Gereksiz bellek alanı kaplamadan büyütülüp küçültülebilir.
- **Kolay yeniden düzenleme:** Hoca örnek olarak tersine çevirmeyi verdi. Bir diziyi tersine çevirmek için $n/2$ kadar yer değiştirme gerekir. Bağlantılı listede ise yalnız işaretçiler bir öncekini gösterecek şekilde güncellenir: D4 D3'ü, D3 D2'yi, D2 D1'i göstersin. Kısacası "beni göstereni göster" kuralıyla liste tersine çevrilir.
- **Bütün indisleri bilmek gerekmez:** Listenin tamamına erişmek için yalnız ilk düğümü gösteren bir referansı, yani ilk düğümün işaretçisini bilmek yeterlidir.
- **Esneklik:** Dairesel (circular) liste, çift yönlü (doubly) liste, çift yönlü dairesel liste gibi, problemin doğasına göre farklı türleri kurulabilir.

4.1 Liste Sonu ve `head`

Her düğümün işaretçisi bir sonraki düğümü gösterir. Son düğümün işaretçisi ise `NULL`'u gösterir, yani hiçbir yeri göstermez. `next` işaretçisi `NULL` olan bir düğüm görüldüğünde, bunun listenin son elemanı olduğu anlaşılır.

Listenin başlangıcını bilmek için özel bir işaretçiye ihtiyaç vardır: `head`. Listeye doğrudan `head` üzerinden başlayarak erişilir. Hoca bir arama örneği verdi: B değerine erişilmek isteniyor. `head`'in gösterdiği düğümde A var. A, B'ye eşit mi? Hayır, bir sonrakine git. Orada B var, eşit. O zaman bulundu. Bağlantılı listede arama bu şekilde çalışır.

4.2 Soyut Veri Tipi ve Kütüphane Önerisi

Bağlantılı liste temelde bir soyut veri tipidir (abstract data type). Kullanıcı tarafından tanımlanır. Veri bölmesinde basit ya da karmaşık veri tipleri barınabilir.

Soyutlamanın amacı gerçekleştirme ayrıntılarını kullanıcıdan gizlemektir. Hoca, bağlantılı listeler için bir kütüphane yazıp fonksiyonlarıyla birlikte kullanmanın kolaylık getireceğini söyledi. Böylece bir işlem yapmak yalnız bir fonksiyon çağrısı olur: eleman ekle, eleman sil, ilk eleman, son eleman gibi. Liste üzerinde neredeyse bir diziymiş gibi kolayca kod yazılır. Her problemde ve her algoritmada liste oluşturma ve eklemeyle satır satır yeniden uğraşmak gerekmez.

5. Bağlantılı Liste Üzerindeki İşlemler

Hoca, bir bağlantılı liste üzerinde yapılabilecek işlemleri tek tek saydı.

Liste oluşturma: Ortada hiç liste yokken önce temel düğüm yapısı (veri ve işaretçi) tanımlanır. Ardından bir `head` işaretçisi yaratılır. Hiç eleman yokken `head` `NULL`'u gösterir. Hoca `NULL`'u elektrikteki "toprak" sembolüyle çizdi. İlk eleman eklenirken yapılacak şey şudur: Liste boş mu? Boşsa bir eleman ekle ve `head`'i bu elemanı gösterecek şekilde güncelle.

Eleman ekleme: Başa eleman eklemek kolaydır ve sabit sayıda işlemle, yani $O(1)$ ile yapılır. Araya eleman eklemek için bir düzene ihtiyaç vardır, örneğin değerlerin büyüklük sırasında durduğu kabulü. Değerler küçükten büyüğe sıralıysa yeni değer ilgili yere eklenebilir. İşin güzel tarafı, yer bulunduktan sonra ekleme işleminin yine sabit sayıda işlem gerektirmesidir. Dizide kaydırma yüzünden $O(n)$ işlem gerekirdi; burada araya ya da sona ekleme $O(1)$ ile yapılır.

Eleman silme: Aynı şey silme için de geçerlidir. Hoca baştan, aradan ya da sondan bir elemanın eleman sayısından bağımsız, sabit sayıda işlemle silinebileceğini söyledi.

> **Ders sonrası not:** Bu ifade, silinecek düğümün öncesi zaten biliniyorsa doğrudur. Tek yönlü listede sondaki düğümü silmek için sondan bir önceki düğüm bulunmalıdır ve bunun için baştan gezmek gerekir. Bu $O(n)$ sürer. Aranan değerin yerini bulmak da doğrusal aramadır.

Listeleri birleştirme: İki liste birleştirilebilir. Ama hoca burada bir soru sordu: İkinci liste doğrudan birincinin sonuna mı eklenecek? Yoksa listeler sıralıysa elemanlar büyüklük sırasına göre birinden bir, öbüründen bir alınarak mı birleşecek? Bu bir tasarım problemidir; bir varsayım yapmak ya da farklı birleştirme biçimleri tanımlamak gerekir.

Listenin boyutunu bulma: İki yol vardır. Birincisi, dışarıda ayrı bir sayaç tutmak; her eklemede bir artırmak, her silmeden sonra bir azaltmak. Bu durumda boyut $O(1)$ ile bulunur. İkincisi, her seferinde `head`'den başlayıp `NULL`'a kadar kaç düğüm geçildiğini saymaktır. Bu $O(n)$ sürer.

Listeyi tersine çevirme: Yalnız bağlantıları tersine çevirerek kolayca yapılır.

> **Ders sonrası not:** Tersine çevirmek kolaydır, ama bütün düğümlerin işaretçisi tek tek güncellenir. Maliyet yine $O(n)$'dir.

Eleman arama: Doğrusal aramadır. Hocaya göre bağlantılı listelerin en zayıf noktası budur. Ortadaki elemanın nerede olduğu bilinmez. İkili arama (binary search) gibi bir şey yalnız `next` bağlantılarıyla yapılamaz. Bu yüzden bağlantılı listelerde arama genellikle doğrusal karmaşıklıktadır.

> **Hocanın vurgusu:** Bağlantılı listenin en zayıf noktası belli bir değerdeki elemanı aramaktır; arama $O(n)$'dir.

6. C'de Düğüm Tanımı ve Listeyi Gezme

Bazı dillerde bağlantılı liste hazır bir veri yapısı olarak bulunur. Standart C'de yoktur, bu yüzden programcı kendisi oluşturur.

6.1 `typedef struct node *link` Tanımı

Hoca önce `struct node` işaretçisi tipinde bir tip tanımladı ve adına `link` dedi. Ardından `node` adında bir yapı tanımladı. İçinde en basit hâliyle bir tamsayı veri ve `link` tipinde `next` vardır:

```c
typedef struct node *link;

struct node {
    int data;
    link next;   /* yani: struct node *next; */
};
```

`link`, `struct node *` demektir. Yani `next` alanı `struct node *next` diye de yazılabilirdi. Bu tanımla her yerde `struct node` tipinde değişken tanımlanabilir. Ancak her kullanımda `struct node` yazmak iki kelimelik ve zahmetlidir.

6.2 Yapının Tamamına `Node` Adı Vermek

Bu yüzden tip tanımı yapının tamamına da yapılabilir:

```c
typedef struct link {
    int data;
    struct link *next;
} Node;
```

Burada `struct link` ile başlayıp kapanan yapının tamamına `Node` adı verilmiştir. İçinde tamsayı bir `data` ve aynı tipten bir işaretçi olan `next` vardır. Hocanın kutucuk çiziminde bir tarafta veri, öbür tarafta `next` vardı; `Node` tam olarak bu kutucuktur. Hoca, `link` ile `Node` arasındaki farkın büyük-küçük harfle net okunduğunu söyledi.

Yeni bir düğüm yaratmak için `link` tipinden bir `x` işaretçisi tanımlanır ve bir düğüm kadar yer ayrılır. `x` bir düğümü göstereceği için işaretçi olmak zorundadır. Böylece bellekte bir kutucuk oluşur ve adresine `x` üzerinden erişilir.

6.3 Listeyi Gezme

`x` yerine bir `head` işaretçisi düşünelim. Listeyi gezmek için `head`'den başlanır, `NULL` olmadığı sürece devam edilir ve her adımda `next`'e gidilir:

```c
for (t = head; t != NULL; t = t->next)
    use(t->data);   /* örneğin printf */
```

Hoca bunu 4 elemanlı bir listeyle adım adım izledi: 3 → 5 → 7 → 8 → `NULL`.

1. `t = head;` `t` 3'ü taşıyan düğümü gösterir. `use(t->data)` 3'ü kullanır, örneğin yazdırır.
2. Artım kısmı `t = t->next;` `t` artık 5'in düğümünü gösterir. `NULL` mı? Hayır. 5 kullanılır.
3. `t = t->next;` `t` 7'yi gösterir. `NULL` değil, 7 kullanılır.
4. `t = t->next;` `t` 8'i gösterir. 8 kullanılır.
5. `t = t->next;` `t` artık `NULL`'dur. Döngü koşulu (`t != NULL`) bozulur ve döngüden çıkılır.

Yani `head`'den `NULL` görene kadar `next`'lerle ilerlenir ve her adımda düğümün verisi alınıp kullanılır.

7. Listeye Eleman Ekleme

7.1 Araya Ekleme

Hoca slayttaki listeyle başladı. `head` ilk düğümü gösteriyor. Değerler sıralı tamsayılardır: 3, 5, 7, 8. Ayrıca `tmp` adında yeni bir düğüm oluşturulmuştur ve içinde `x` değeri vardır. Bu düğüm A ile B adlı iki düğümün arasına eklenecektir.

Bunu yapmak için A'nın `next`'i koparılıp `x`'i gösterecek şekilde güncellenmeli, `x`'in `next`'i de A'nın eski `next`'ini göstermelidir. A'yı gösteren işaretçiye `current` diyelim; bunun `head` olması gerekmez.

Soru (hoca): İlk iş olarak `current->next`'i `tmp` yapsak ne olur? **Cevap:** Bunu yaptığımız anda B'yi, yani `current`'ın eski `next`'ini kaybederiz. Listenin geri kalanı erişilemez hâle gelir.

Bu yüzden sıra şudur:

```
tmp->next = current->next;   /* 1. adım */
current->next = tmp;         /* 2. adım */
```

Slayttaki gösterimle aynı şey `x->next = a->next; a->next = x;` biçimindedir. Bu iki atama yapıldığında listenin sonrası kaybedilmeden araya bir eleman eklenmiş olur.

> **Hocanın vurgusu:** Araya eklemede önce yeni düğümün `next`'i bağlanır, sonra öncekinin `next`'i yeni düğüme çevrilir. Sıra ters olursa listenin geri kalanı kaybolur.

7.2 Başa Ekleme

Yeni eleman ilk eleman olarak eklenecekse yapılacak tek şey `head`'i güncellemektir:

```
tmp->next = head;
head = tmp;
```

Önce `tmp`'nin `next`'i `head`'in gösterdiği düğümü gösterir. Sonra `head`, `head = tmp` ile koparılıp yeni düğümü gösterecek şekilde güncellenir. Böylece başa eleman eklenmiş olur.

7.3 Sona Ekleme

Sona eklemek için `NULL` görene kadar gidilir. `tmp`'nin `next`'i `NULL` yapılır, son düğümün `next`'i de `tmp` yapılır.

8. İki Dizi ile Bağlantılı Liste ve Sıralı Listeye Ekleme

Ara sonrası hoca, sıralı bir bağlantılı listeye eleman eklemeyi ve bunun akış diyagramıyla nasıl gösterileceğini anlattı.

8.1 Listeyi İki Diziyle Göstermek

Bağlantılı listeler işaretçi yapısına sahip olduğu için akışta birebir göstermek kafa karıştırıcı olabilir. Ama n elemanlı bir listede n tane değer ve (`head` hariç) n tane link değeri vardır. Bunlar dizi gibi gösterilebilir. Hatta yalnız iki dizi kullanarak bağlantılı liste gerçekleştirilebilir:

- `A`: veriler (data),
- `L`: her elemanın bir sonrakinin indisi (link),
- `head`: ilk elemanın indisi.

Böyle bir listeyi okumak için dört şey yeterlidir: n, değerler, linkler ve `head`. Ekleme akışında bunlara ek olarak eklenecek değer (`ekle`) okunur. Liste sonu `-1` ile gösterilir; böylece `NULL`'a gelindiği kolayca takip edilir.

8.2 Sıralı Listeye Ekleme Akışı

1. Yeni değer dizinin sonuna yazılır ve eleman sayısı artırılır: `A[n] ← ekle`, `n ← n + 1`. Yeni elemanın indisi artık `n-1`'dir.
2. `A[head] < ekle` değilse, yeni eleman en küçüktür ve başa girer: `L[n-1] ← head`, `head ← n-1`.
3. Değilse ilerlemek gerekir. İki işaretçi tutulur: `prev ← head`, `next ← L[head]`. Bu ikisi tutulur, çünkü yeni eleman bu ikisinin arasına eklenecektir.
4. `next ≠ -1` (son elemana gelinmedi) ve `A[next] < ekle` olduğu sürece `prev ← next`, `next ← L[next]` ile ilerlenir.
5. Döngüden çıkınca linkler güncellenir: `L[prev] ← n-1`, `L[n-1] ← next`.

Hoca 01:03:39'da slayttaki `L[prev] ← n-1` ifadesini önce "n olacak, yanlış yazmışız" diye düzeltmeye kalktı. Sonra "`n ← n+1` dedik, `n-1` doğru" diyerek geri aldı. Bu indis yazımı 0 tabanlı dizilere uyar: `A[n]` dizinin ilk boş gözüdür.

Hoca ayrıca sona eklemenin ayrı bir durum gerektirmediğini söyledi: Eklenen değer hepsinden büyükse `next = -1` koşuluyla döngüden çıkılır ve eleman sona eklenir.

8.3 Birinci Örnek: 1 Tabanlı Dizilerle 6 ve 1 Ekleme

Hoca 3, 5, 7, 8 değerlerini dizide karışık sırayla yerleştirdi. Akış olduğu için indisler 1'den başlar.

İndis	1	2	3	4
A	7	5	3	8
L	4	1	2	-1

head = 3. Gezinti: A[3] = 3 → L[3] = 2 → A[2] = 5 → L[2] = 1 → A[1] = 7 → L[1] = 4 → A[4] = 8 → L[4] = -1, dur. Liste 3 → 5 → 7 → 8'dir.

6 eklenmesi:

1. Eleman sayısı bir artırılır ve 5. göze 6 yazılır.
2. A[head] = A[3] = 3. 3 < 6 olduğu için başa eklenmez, ilerlenir.
3. prev = head = 3, next = L[3] = 2.
4. next ≠ -1 ve A[2] = 5 < 6 → prev = 2, next = L[2] = 1.
5. next ≠ -1 ama A[1] = 7 < 6 değil → döngüden çıkılır.
6. L[prev] = L[2] ← 5 (yeni elemanın indisi), L[5] ← next = 1.

Hoca burada bir an L[2]'ye değeri (6) yazdı, sonra "değeri yazdık yanlışlıkla, 5. indisi gösteriyorum" diye düzeltti. Link dizisine her zaman indis yazılır.

İndis	1	2	3	4	5
A	7	5	3	8	6
L	4	5	2	-1	1

head değişmedi (3). Gezinti: 3 → (2'ye git) 5 → (artık 1 yerine 5'e git) 6 → (1'e git) 7 → (4'e git) 8 → -1, dur. Sonuç: 3, 5, 6, 7, 8.

1 eklenmesi (başa ekleme):

1. 6. göze 1 yazılır.
2. A[head] = A[3] = 3. 3 < 1 sağlanmaz, yani yeni eleman en küçüktür.
3. Yeni elemanın linki eski head olur: L[6] ← 3. head ← 6.

İndis	1	2	3	4	5	6
A	7	5	3	8	6	1
L	4	5	2	-1	1	3

head = 6. Gezinti: 1 → 3 → 5 → 6 → 7 → 8. Akış yalnız iki şeyi aynı anda kontrol eder: "hangisinden küçük, hangisinden büyük?" ve "sona geldim mi?". Önceki ve sonraki linkler tutulur; öncekinin linki yeni elemana, yeni elemanın linki de öncekinin eski sonrasına çevrilir.

8.4 İkinci Örnek: 0 Tabanlı Dizilerle 1 ve 5 Ekleme

Slayttaki ikinci örnekte değerler sıralı yerleşmişti ve indisler 0'dan başlıyordu.

İndis	0	1	2	3
A	2	4	6	8
L	1	2	3	-1

head = 0, $n = 4$. Liste 2 → 4 → 6 → 8.

1 eklenmesi: 4. göze 1 yazılır ve n 5 olur. `A[head] = A[0] = 2`; $2 < 1$ sağlanmaz, eleman başa girer. 4 numaralı gözün linki `head` yani 0 olur, `head` ise $n - 1 = 4$ olur. Liste artık $4 \to 0 \to 1 \to 2 \to 3 \to$ `-1` indis sırasıyla gezilir: 1, 2, 4, 6, 8.

5 eklenmesi: 5. göze 5 yazılır ve n 6 olur.

Adım	`prev`	`next`	Kontrol	Karar
Başlangıç	4	`L[4] = 0`	`A[head] = 1 < 5`	ilerle
1	4	0	`A[0] = 2 < 5`	`prev = 0`, `next = 1`
2	0	1	`A[1] = 4 < 5`	`prev = 1`, `next = 2`
3	1	2	`A[2] = 6 < 5` değil	döngüden çık

Çıkışta `L[prev] = L[1] ← 5`, `L[5] ← next = 2`.

İndis	0	1	2	3	4	5
`A`	2	4	6	8	1	5
`L`	1	5	3	-1	0	2

`head = 4`. Gezinti: $1 \to 2 \to 4 \to 5 \to 6 \to 8$. Böylece araya eleman eklenmiş oldu.

9. Ekleme Fonksiyonlarının C Kodu

9.1 Başa Ekleme: `insert_top`

Hoca bu kez yapıya büyük harfle `Node` adını verdiğine dikkat çekti (Bölüm 6.2'deki tanım). Fonksiyon geriye bir `Node *`, yani listenin başının (`head`) adresini döndürür. Adı `insert_top` ya da `insert_head` olabilir. İki parametre alır: düğümün verisine yazılacak değer ve `head` işaretçisi.

```
Node *insert_top(int num, Node *head) {
    Node *new_node = (Node *)malloc(sizeof(Node));
    new_node->data = num;
    new_node->next = head;
    head = new_node;
    return head;
}
```

Hoca adımları sorularla ilerletti:

1. İlk eleman eklemek için önce bir düğüm oluşturmak gerekir: `new_node`, `malloc` ile `sizeof(Node)` kadar yer ayrılarak oluşturulur ve `Node *`'a dönüştürülür (cast).
2. Düğümün verisine gelen `num` yazılır.
3. `new_node`'un `next`'i eski `head`'i gösterecek şekilde bağlanır.

Soru (hoca): İşim bitti mi? **Cevap:** Bitmedi. Düğüm listenin başına bağlandı, ama `head` hâlâ eski ilk düğümü gösteriyor. `head = new_node` ile güncellenmelidir.

Soru (hoca): Şimdi bitti mi? **Cevap:** Hayır. Fonksiyonun geri dönüş değeri sağlanmalıdır. Listenin başı `return head` ile çağırana döndürülür.

9.2 Belirli Bir Değerin Önüne Ekleme: `insert_before`

Hoca daha genel bir ekleme fonksiyonu yazmak istedi: belli bir değerin, yani belli bir düğümün önüne ekleme. Örneğin 3, 5, 8 listesinde "8'den önceye ekle". Liste sıralı olmak zorunda değildir. Bu fonksiyon `head`'i güncellemez, ama yine `Node *` döndürür.

Fikir şudur: `current` düğümünün `next`'inin verisi, verilen `next_num`'a eşit olmadığı sürece ilerlenir. Neden `current`'ın kendisine değil de `next`'inin verisine bakılır? Çünkü yeni düğüm aranan düğümün **önüne** eklenecektir. Ekleme için aranan düğümün bir öncesinin bilinmesi gerekir. Eşitlik sağlandığı anda ekleme koduna geçilir.

```
Node *insert_before(int next_num, int num, Node *head) {
    Node *cur = head;
    while (cur->next->data != next_num)
        cur = cur->next;
    Node *new_node = (Node *)malloc(sizeof(Node));
    new_node->data = num;
    new_node->next = cur->next;   /* 1. adım */
    cur->next = new_node;         /* 2. adım */
    return head;
}
```

Dikkat edilmesi gereken nokta: Yeni eleman ilk eleman olarak eklenecekse bu kontrol çalışmaz. Bu yüzden `insert_top` ayrı bir fonksiyon olarak tanımlandı. Hoca, isteyenlerin daha düzgün bir kodla iki fonksiyonu birleştirebileceğini söyledi ve bunu öğrencilere bıraktı: Eliniz alışsın, bir bağlantılı liste kütüphaneniz oluşsun, ilerideki ödevlerinizde de kullanırsınız.

> **Ders sonrası not:** Aranan değer listede yoksa `cur->next` sonunda `NULL` olur ve `cur->next->data` erişimi programı çökertir. Döngü koşuluna `cur->next != NULL` kontrolü eklenmelidir: `while (cur->next != NULL && cur->next->data != next_num)`. Döngüden `cur->next == NULL` ile çıkılırsa değer bulunmamıştır.
>
> **Ek bilgi (derste anlatılmadı):** `insert_top` ve `insert_before`'u birleştirmenin yaygın bir yolu, listenin başına veri taşımayan bir "kukla" (dummy, sentinel) düğüm koymaktır. Böylece ilk elemanın da bir öncesi olur ve başa ekleme ayrı bir durum olmaktan çıkar. Diğer yol, fonksiyona `Node **` (head'in adresi) vermektir.

10. Listeden Eleman Silme

10.1 Temel Fikir ve `free`

Elimizde A → B → C diye giden bir liste olsun ve ortadaki B silinmek istensin. `current` A'yı göstersin. Yapılması gereken, `current`'ın `next`'ini `next`'inin `next`'ine bağlamaktır:

```
current->next = current->next->next;
```

Bu yazılabilir, çünkü `current->next` yine bir düğüm işaretçisidir ve onun da `next`'i vardır. Bu atamadan sonra A'dan `next` denince C'ye gidilir. B artık listede değildir ve ona erişilemez.

Hoca bunun en basit yol olduğunu, ama iyi bir yöntem olmadığını söyledi. Çünkü B düğümü bellekte yer kaplamaya devam eder. Program sonlanıp bütün bellek alanlarını işletim sistemine geri verene kadar, bir düğüm kadar alan ayrılmış ama hiçbir şekilde erişilemez hâlde kalır. Bu kötü bir gerçekleştirmedir.

Doğrusu, silinecek düğümü önce bir değişkende tutmaktır. Hoca önce `tmp` adını kullandı, sonra daha kolay okunsun diye `b` dedi:

```
b = a->next;        /* silinecek düğüm */
a->next = b->next;  /* bağlantıyı atla (current->next->next de yazılabilir) */
free(b);            /* belleği geri ver */
```

`free(b)` ile ayrılan alan serbest bırakılır ve işletim sistemine "bu alanı başka süreçler için kullanabilirsin" denmiş olur. Böylece silme üç adımda yapılır.

> **Hocanın vurgusu:** Listeden çıkarılan düğüm mutlaka `free` edilmelidir. Aksi hâlde program bitene kadar bellekte erişilemeyen bir alan kalır.

10.2 İki Diziyle Sıralı Listeden Silme Akışı

Akışta yine bir `A` veri dizisi ve bir `L` link dizisi kullanılır. İkisi de n boyutludur (`1..n` ya da `0..n-1`). Ayrıca silinecek değer (`sil`) okunur. Bu bir dizi gösterimi olduğu için silmede dizinin boyutu küçültülmez.

1. Silinecek eleman ilk elemansa (`A[head] = sil`), yapılacak tek şey `head`'i güncellemektir: `head ← L[head]`. İlk eleman böylece erişilemez hâle gelir.

2. Değilse, eklemede olduğu gibi bir önceki ve bir sonraki tutularak ilerlenir: `prev ← head`, `adres ← L[head]`. Hoca "adres" adını silinecek adres anlamında verdi; `next` de denebilirdi.
3. `adres ≠ -1` ve `A[adres] < sil` olduğu sürece `prev ← adres`, `adres ← L[adres]`.
4. Döngüden çıkınca bir kontrol daha yapılır: `adres = -1` (sona gelindi) ya da `A[adres] > sil` ise aranan eleman listede yoktur, "listede yok" hatası döndürülür.
5. Bu koşul yanlışsa ya son eleman ya da aradaki bir eleman silinecektir: `L[prev] ← L[adres]`. Yani öncekinin linki, silinenin linki yapılır ve aradaki eleman erişilemez olur.

Hoca isteğe bağlı bir ayrıntı da ekledi: Silinen elemanın link gözüne `-2` gibi özel bir kod yazılabilir. Böylece link dizisinde hangi elemanların silindiği takip edilir. Bunun için `head` güncellenmeden önce eski değeri tutulmalıdır.

10.3 Örnek: 2 ve 6'nın Silinmesi

Değerler sıralı, indisler 0 tabanlı, $n = 6$, `head = 0`:

İndis	0	1	2	3	4	5
`A`	2	3	4	6	7	8
`L`	1	2	3	4	5	-1

Gezinti: 2 → 3 → 4 → 6 → 7 → 8 → son.

2 silinmesi: `sil = 2`. `A[head] = A[0] = 2`, eşit. İlk değer silinecek. `head ← L[0] = 1`. Artık listeye 1. gözden başlanır.

6 silinmesi: `head = 1`.

Adım	`prev`	`adres`	Kontrol	Karar
Başlangıç	1	`L[1] = 2`	`A[1]` = 3, 6 değil	ilerleme başlar
1	1	2	`adres ≠ -1`, `A[2]` = 4 < 6	`prev` = 2, `adres` = 3
2	2	3	`A[3]` = 6 < 6 değil	döngüden çık

Çıkış kontrolü: `adres = -1` mi? Hayır. `A[3] = 6 > 6` mı? Hayır. O hâlde eleman bulundu: `L[prev] = L[2] ← L[3] = 4`.

İndis	0	1	2	3	4	5
`A`	2	3	4	6	7	8
`L`	1	2	4	4	5	-1

`head = 1`. Gezinti: 3 → (2'ye git) 4 → (eskiden 3'e gidiyordu, artık 4'e) 7 → (5'e git) 8 → `-1`. Kalan liste 3, 4, 7, 8'dir.

10.4 Silmenin C Kodu

Fonksiyon silinecek değeri ve `head`'i alır, `head`'i döndürür. `current` `head`'den başlar, ayrıca bir `previous` işaretçisi tutulur. Hoca slaytta bir sayaç (counter) değişkeni de bulunduğunu, ama kullanılmadığını belirtti. `current` `NULL` olmadığı sürece ilerlenir. Düğümün verisi silinecek değere eşitse iki durum vardır:

- `current` `head` ise: `head = current->next`.
- Değilse: `previous->next = current->next`.

Eşit değilse `previous = current`, `current = current->next` ile bir adım kaydırılır.

Hoca slayttaki kodda `free` bulunmadığını söyledi. Listeden atılan düğüm bellekte yer kaplamaya devam eder. Her iki durumdan sonra `current` `free` edilmelidir. Hoca bu yere bir ünlem işareti koydu. `free` eklenmiş hâli şöyledir:

```c
Node *delete_value(int num, Node *head) {
    Node *cur = head, *prev = NULL;
    while (cur != NULL) {
        if (cur->data == num) {
            if (cur == head)
                head = cur->next;
            else
                prev->next = cur->next;
            free(cur);   /* slaytta eksikti, hoca eklenmesini söyledi */
            return head;
        }
        prev = cur;
        cur = cur->next;
    }
    return head;
}
```

> **Ders sonrası not:** `free(cur)` eklendikten sonra döngü devam ederse `cur = cur->next` serbest bırakılmış belleğe erişir. Bu yüzden yukarıdaki kodda silmeden sonra hemen `return head` yapılmıştır. Değerin birden fazla kopyası silinecekse, `free`'den önce `cur->next` ayrı bir değişkende saklanmalıdır.

11. Bağlantılı Liste ile Insertion Sort

Son saatte hoca, bağlantılı listeyle bir algoritma örneği verdi: herkesin bildiği insertion sort. Soru şudur: Insertion sort bağlantılı listeyle çalıştırılabilir mi, nasıl?

Elemanlar karışık sırada bir `A` bağlantılı listesinde dursun; örneğin 627, 758, 113 gibi değerler. Bunları sıralamak için ikinci bir liste kullanılır: çıktı listesi `B`. Hoca, aynı liste üzerinde iki `head` işaretçisiyle (biri yeni, biri eski baş) de yapılabileceğini, ama iki listeyle slaytta göstermenin daha kolay olduğunu söyledi.

Her yinelemede:

1. Girdi listesinin başındaki düğüm koparılır. Bu $O(1)$'dir.
2. Düğüm çıktı listesinde olması gereken yere eklenir. Bu aslında bir `insert_before` ya da `insert_after` işlemidir.

Örnek akış:

- **627:** `B`'nin başından başlanır. 98 < 627, ilerle. 101 < 627, ilerle. 515 < 627, ilerle. 838 > 627, dur. 627 515 ile 838 arasına girer: önce `tmp->next = current->next` (1. adım), sonra `current->next = tmp` (2. adım). Girdi listesinin başı da 758'i gösterecek şekilde güncellenir.
- **758:** 101, 515, 627 geçilir, 838'de durulur. 758, 627 ile 838 arasına eklenir.
- **113:** Uygun araya (101 ile 515 arasına) eklenir.

Böylece ikinci listede sıralı bir sonuç oluşur. Problem "verilen bir bağlantılı listeyi insertion sort ile sıralamak" diye de okunabilir. Dizideki gibi kaydırma yapılmaz; yalnız bağlantılar koparılıp bağlanır. Hoca, selection sort gibi diğer sıralamalar için de bir şeyler düşünülebileceğini, ama onlarda bağlantılı listenin pek mantıklı olmayacağını söyledi.

> **Ders sonrası not:** Bağlantılı liste kaydırmayı ortadan kaldırır, ama her eleman için çıktı listesinde baştan doğrusal arama yapılır. En kötü durumda toplam karşılaştırma sayısı yine $O(n^2)$'dir.

12. Çift Yönlü Bağlantılı Liste

Bağlantılı listelerin en sık kullanılan türlerinden biri çift yönlü bağlantılı listedir (doubly linked list). Burada her düğümde iki işaretçi vardır: biri ileri (`next`), biri geri (`prev`). `next`'ler üzerinden sona doğru, `prev`'ler üzerinden başa doğru ilerlenebilir.

Böyle bir listede iki özel işaretçi gerekir. Birincisi bilinen head'dir. İkincisi tail'dir; son elemanı, yani next'i NULL olan elemanı gösterir. tail'den başlayıp prev'lerle head'e kadar geriye doğru gezinti yapılabilir.

```
typedef struct link {
    int data;
    struct link *next;
    struct link *prev;
} Node;
```

12.1 Esneklik: Ağaç, Matris, Veri Küpü

Hoca, next'lerin sağa, prev'lerin sola gittiği gösterimin yalnız bir varsayım olduğunu söyledi. Yapı çok esnektir. İlerideki haftalarda ağaçlarda görüleceği gibi, bir işaretçi bir sonraki kardeşi, öbürü ilk çocuğu gösterebilir. Bir bağlantı "aşağı", öbürü "sağ" olabilir. Bununla bir matris yapısı bile kurulabilir: Her düğüm sağındakini ve altındakini gösterir; satır ve sütun sonları NULL'dur. Üç işaretçiyle (triple linked list) bir veri küpü oluşturulabilir. Ancak bu işaretçilerde sağa, aşağı, ileri, geri gibi hareketleri kodlarken tek yönlü listelere göre daha dikkatli olmak gerekir.

12.2 Araya Ekleme: Dört Adım

Tek yönlü listede araya eklemek için bir bağlantı iki bağlantıya çevrilir ve bunun için iki işlem yapılır. Çift yönlü listede hem next hem prev vardır. Yani iki bağlantıyı dört bağlantıya çevirmek, dört işlem yapmak gerekir. Bunlar önceki ve sonraki adresleri kaybetmeden belli bir sırayla yapılmalıdır.

current bir düğümü göstersin. Onun next'i sonraki düğüme gider, sonraki düğümün prev'i current'a döner. malloc ile new adında yeni bir düğüm yaratılmıştır; prev ve next alanları şimdilik NULL'dur. new, current ile sonraki düğümün arasına eklenecektir.

```
new->next = current->next;          /* 1. adım */
current->next->prev = new;          /* 2. adım */
current->next = new;                /* 3. adım */
new->prev = current;                /* 4. adım */
```

1. new'in next'i current'ın next'ine bağlanır. Bu adres zaten bilinmektedir.
2. Sonraki düğümün geri dönen bağlantısı güncellenir: Eskiden current'ı gösteriyordu, artık new'i gösterir.
3. current'ın next'i artık new'i gösterir.
4. new'in prev'i current olur.

Sonuçta current->next ile new'e, onun next'i ile sonraki düğüme gidilir. Sonraki düğümün prev'i new'e, new'in prev'i de current'a döner. Özetle önce yeninin next'i, sonra sonrakinin prev'i, sonra current'ın next'i ve yeninin prev'i güncellenir.

> **Hocanın vurgusu:** Çift yönlü listede araya eklemek için iki bağlantı, dört işlemle güncellenir. Bu dört işlem listenin öncesini ve sonrasını kaybetmeyecek sırayla yapılmalıdır.

12.3 Silme: Üç Adım

del işaretçisi silinecek düğümü göstersin. Düğümün dört bağlantısı vardır. Amaç, öncekinin next'inin silinenin sonrasını, sonrakinin prev'inin de silinenin öncesini göstermesidir.

```
del->next->prev = del->prev;    /* 1. adım */
del->prev->next = del->next;    /* 2. adım */
free(del);                      /* 3. adım */
```

1. Silinecek elemanın sonrasındaki elemanın prev'i, del'in prev'ini gösterir. Sonrakilerden prev ile del'e gelme yolu kesilir.
2. del'in öncesindeki düğümün next'i, del'in next'ini gösterir. Öncekilerden next ile del'e gelme yolu kesilir.
3. free(del) ile düğüm bellekten gerçekten silinir.

`del` henüz `free` edilmeden önce de iki komşusunu göstermeye devam eder. Bu yüzden 1. ve 2. adımlarda `del->prev` ve `del->next` güvenle kullanılır. Hoca, bu ekleme ve silme işlemlerinin kodunu öğrencilerin mutlaka kendilerinin yazmasını önerdi.

> **Ek bilgi (derste anlatılmadı):** Yukarıdaki dört ve üç adımlık kodlar, `current`'ın sonrasında ve `del`'in iki yanında bir düğüm olduğunu varsayar. `current` son düğümse `current->next->prev` ya da `del` baş/son düğümse `del->prev->next` / `del->next->prev` `NULL` işaretçiye erişir. Gerçek kodda bu durumlar `NULL` kontrolüyle ayrıca ele alınır ve gerekirse `head` ya da `tail` güncellenir.

13. Dairesel Bağlantılı Liste ve Josephus Problemi

13.1 Dairesel Liste

İşaretçi aritmetiğinin esnekliği sayesinde bağlantılar istenildiği gibi kurulabilir: öncesini, sonrasını, çocuğunu, kardeşini, matristeki alt satırı, yan komşuyu göstersin. Dersin son başlığı dairesel bağlantılı listedir (circular linked list).

Hocanın örneği sıralı bir listeydi: 2, 5, 7, 8, 10. `head` ilk düğümü gösteriyor. Normalde son düğümün işaretçisi `NULL`'u gösterir. Son düğümün işaretçisi `NULL` yerine `head`'i gösterecek şekilde güncellenirse ne olur? Kendi üzerine dönen, dairesel bir liste oluşur.

Bu listede `head`'den başlayıp `next`, `next`, `next` denince sona gelinir, bir `next` daha denince yine başa dönülür. Hoca bunu çamaşır makinesine ya da tekerleğe benzetti: sürekli aynı yerde döner. Sonda `NULL` diye bir şey yoktur, `head` vardır. Bu yüzden ekleme ve silmede `NULL` kontrolü yapılamaz. Son elemanı bulmak için `head`'den ikinci kez geçilip geçilmediği kontrol edilmelidir.

13.2 Josephus (Romalı) Problemi

Hoca dairesel listeyle çözülebilecek, tarihin eski zamanlarından kalma bir problemden bahsetti. Türkçede "Romalı problemi", literatürde Josephus problemi olarak bilinir. Hocanın anlattığı hikâye şöyledir:

Roma İmparatoru Nero (Roma'yı yakan imparator olarak da bilinir) zamanında Orta Doğu'da bir Yahudi ayaklanması çıkar. Nero, ayaklanmayı bastırmak için general Vespasianus'u görevlendirir. O bölgede ünlü bir Yahudi tarihçi vardır: Flavius Josephus. Problemin adı buradan gelir. Söylenceye göre Josephus ve 40 arkadaşı askerlerden kaçar ve bir mağarada kıstırılır; teslim olmak zorunda kalırlar. Vespasianus, toplam 41 esirin çember şeklinde dizilmesini ve her üçüncü kişinin öldürülmesini emreder: 1, 2 sayılır, 3 öldürülür; 4, 5 sayılır, 6 öldürülür; 7, 8 sayılır, 9 öldürülür. Bu, $m - 1 = 3 - 1 = 2$ kişi kalana kadar sürecektir.

Kimse 3, 6, 9 gibi numaraları almak istemez, çünkü bunlar ilk turda ölecektir. Ama general gaddardır, kimse yerinden kıpırdayamaz. Bu arada Josephus hızla bir hesap yapar: 41 kişilik ve her üçüncünün öldürüldüğü dizilimde hangi iki numaranın sona kalacağını bulur. Kendisini bir yere, en sevdiği arkadaşını öbür numaraya koyar. Herkes ölür, son iki kişi Josephus ve arkadaşıdır. Hikâyeye göre Josephus, Vespasianus'a Nero'dan sonra Roma imparatoru olacağını söyler ve canı bağışlanır. Hoca hikâyenin bir kısmının gerçek, bir kısmının abartı olabileceğini söyledi.

> **Ders sonrası not:** Hoca olayın "milattan önce" bir zamanda geçtiğini söyledi. Olay büyük olasılıkla MS 67 yılında, Roma'ya karşı Birinci Yahudi İsyanı sırasında (Yodfat/Jotapata kuşatması) geçmiştir. Yaygın anlatımda da büyük olasılıkla sayma ve öldürme işini Romalılar değil, teslim olmak istemeyen esirler kendi aralarında yapmıştır.

Problemin biçimsel hâli şöyledir: n elemanlı bir çemberde her yinelemede m'inci eleman siliniyorsa, sona kalan $m - 1$ eleman hangileridir (ya da sona kalan tek eleman hangisidir)?

13.3 Diziyle ve Dairesel Listeyle Çözüm Fikri

Hoca basit olsun diye $n = 9$, $m = 5$ aldı. Algoritma n ve m'in bütün değerleri için çalışır.

Dizi ile yapılsaydı: 1'den 9'a kadar sayılar bir dizide tutulur. 1, 2, 3, 4, 5 sayılır, 5. eleman -1 ya da 0 yapılarak işaretlenir. Sonra 1, 2, 3, 4 sayılır ve dizinin sonuna gelinir. Sona gelince

mod alınarak başa dönülür, sayaç sıfırlanır ve devam edilir. Silinmiş gözlerin atlanması, başa dönme gibi sürekli kontroller gerekir.

Dairesel listeyle: 1'den 9'a kadar düğümler sırayla bağlanır, 9'un `next`'i `NULL` yerine `head`'i (1'i) gösterir. `head`'den başlanır ve 5'e kadar sayılır. Bir kişiyi "öldürmek", öncekinin `next`'ini onun `next`'i yapmaktan ibarettir.

13.4 $n = 9$, $m = 5$ için Adım Adım Çözüm

Tur	Sayılanlar	Elenen	Bağlantı güncellemesi	Kalanlar
1	1, 2, 3, 4, 5	5	4 → 6	1 2 3 4 6 7 8 9
2	6, 7, 8, 9, 1	1	9 → 2	2 3 4 6 7 8 9
3	2, 3, 4, 6, 7	7	6 → 8	2 3 4 6 8 9
4	8, 9, 2, 3, 4	4	3 → 6	2 3 6 8 9
5	6, 8, 9, 2, 3	3	2 → 6	2 6 8 9
6	6, 8, 9, 2, 6	6	2 → 8	2 8 9
7	8, 9, 2, 8, 9	9	8 → 2	2 8
8	2, 8, 2, 8, 2	2	8 → 8	8

Son $m - 1 = 4$ kişi kalınca durulursa 2, 6, 8, 9 hayatta kalır. Hoca "Vespasianus'tan daha gaddar olup" tek kişi kalana kadar devam etti: 6, 9 ve 2 de elenir. Son elemanda 8'in `next`'i kendisine bağlanır. Sona kalan kişi 8'dir. (Elenme sırası 5, 1, 7, 4, 3, 6, 9, 2'dir; python ve C ile doğrulandı.)

13.5 Josephus Kodu

Hoca kodun yalnız gövdesini gösterdi; `main` ve başlık kısımları yazılmamıştı. Kod `link` tip tanımını (Bölüm 6.1) kullanır:

```
int i, N = 9, M = 5;
link t = malloc(sizeof *t), x = t;

t->data = 1; t->next = t;

for (i = 2; i <= N; i++) {
    x = (x->next = malloc(sizeof *x));
    x->data = i; x->next = t;
}

while (x != x->next) {
    for (i = 1; i < M; i++) x = x->next;
    x->next = x->next->next;
    N--;
}
printf("%d\n", x->data);
```

İlk kısım dairesel listeyi oluşturur. Hocanın benzetmesiyle bu, esirlerin el ele tutuşup çember oluşturmasıdır. `t` ilk düğümdür; verisi 1'dir ve `next`'i kendisidir. Sonra 2'den N'ye kadar her değer için bir düğüm yaratılır, önceki düğüme bağlanır ve `next`'i başa (`t`'ye) ayarlanır. Döngü bittiğinde `x` son düğümü (9) gösterir.

İkinci kısım eleme döngüsüdür. `x`, `x->next`'e eşit olmadığı sürece, yani tek eleman kalana kadar, `x = x->next` ile $M - 1$ adım ilerlenir. Ardından `x`'in sonrasındaki düğüm, yani sayılan M'inci kişi, listeden çıkarılır. N bir azaltılır. Son kalan düğümün verisi yazdırılır.

> **Ders sonrası not (karmaşıklık):** Hoca toplam maliyeti şöyle hesapladı: oluşturma $O(n)$, silme kısmı $n-m$ eleman; toplam $O(\text{sabit} + n + (n-m)) = O(2n)$, yani $O(n)$. Ancak her eleme için $m - 1$ adım ilerleniyor ve tek kişi kalana kadar $n - 1$ eleme yapılıyor ($m - 1$ kişide durulursa $n - m + 1$ eleme). Doğru toplam $O(n \cdot m)$'dir. m sabit kabul edilirse $O(n)$'e iner.

> **Ders sonrası not (bellek):** Kod, listeden çıkarılan düğümleri `free` etmiyor. Gerçek bir uygulamada çıkarılan düğüm önce bir değişkende tutulmalı, bağlantı atlandıktan sonra `free` edilmelidir (Bölüm 10.1).

Hoca bu örnekle şunu vurguladı: Normalde başa geldin mi, sona geldin mi, kaç saydın gibi pek çok kontrol gerekirken, dairesel liste ile hiç `if` kullanmadan, tek bir döngüyle çözüm elde edildi. Veri yapıları algoritmaların içinde çözümü kolaylaştıran malzeme olarak destek olur.

> **Hocanın vurgusu:** Uygun veri yapısı algoritmayı sadeleştirir. Josephus problemi dairesel listeyle, karar mekanizması kullanmadan çözülür.

13.6 Dairesel Çift Yönlü Liste ve Seçim Kuralı

Hoca hepsini bir araya getiren bir yapı da gösterdi: dairesel çift yönlü bağlantılı liste (circular doubly linked list). Düğümlerde veri vardır, bir başlangıç (start) işaretçisi vardır. `next`'lerle ileri, `prev`'lerle geri gidilir. Son düğümün `next`'i `NULL` yerine başa, ilk düğümün `prev`'i sona gider. Sürekli bir ileri bir geri dönmeniz gerekiyorsa böyle bir yapı işe yarar.

Hocanın özeti:

- Tek yönde gidilecekse tek yönlü liste (singly linked list) yeter.
- Hem ileri hem geri hareket gerekiyorsa ve bu işi hızlandıracaksa çift yönlü liste daha mantıklıdır.
- Bir yerde durmak yerine sürekli başa dönmek gerekiyorsa dairesel liste daha uygundur.

14. Dizi mi, Bağlantılı Liste mi?

Son slaytta hoca, bağlantılı listenin her zaman daha iyi olmadığını, bazen dizinin bazen listenin uygun olduğunu söyledi ve karşılaştırma yaptı:

Durum	Uygun yapı	Gerekçe
Ekleme ve silme hep sonda yapılıyor	Dizi	Araya ekleme yoksa kaydırma da yoktur; statik bir yapı yeter.
Başa ya da araya sık ekleme/silme var	Bağlantılı liste	Çok daha esnek ve daha az maliyetli.
Elemana indisle doğrudan erişim gerekiyor	Dizi	İndisle "tak diye" erişilir.
Rastgele erişim gerekmiyor, veriler sırayla kullanılıyor	Bağlantılı liste	`next`, `next` diye ilerlenir.
Belli bir değer aranıyor ve veri sıralı	Dizi	İkili arama ile $O(\log n)$.
Eleman sayısı belli değil ya da veri karışık, her hâlükârda doğrusal arama gerekiyor	Bağlantılı liste	Arama zaten doğrusal.

Bellek: n elemanlı bir tamsayı dizisi kabaca n birim yer kaplar; alan karmaşıklığı $O(n)$'dir. Bağlantılı listede her düğümde hem veri hem link tutulur. Bu yüzden kabaca $2n$ birim yer kaplanır. Büyük O açısından $2n$ yine $O(n)$'dir. Ama gerçekte iki katına yakın yer kaplanır. Hoca bunu iki dizili gösterimde (`A` ve `L`) de görmüştük diye hatırlattı ve bunu bağlantılı listelerin ekstra bir handikapı olarak saydı. Yine de aralara dinamik olarak çok ekleme ve silme yapılacaksa bağlantılı liste daha mantıklıdır.

> **Ek bilgi (derste anlatılmadı):** Gerçekte bir düğümün boyutu, verinin ve işaretçinin boyutuna bağlıdır. 64 bitlik bir sistemde 4 baytlık `int` ve 8 baytlık işaretçi, hizalama (padding) ile birlikte büyük olasılıkla 16 bayt eder; ayrıca `malloc`'un kendi yönetim bilgisi de eklenir. "Yaklaşık $2n$" bu yüzden kaba bir tahmindir.

Örnek Sorular

Bu derste hazır kare ve video kullanılmadı. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:41:20): Tek yönlü listede `current` ile sonraki düğüm arasına `tmp` eklenirken ilk iş olarak `current->next = tmp` yapılırsa ne olur? Doğru sıra nedir?

> **Çözüm (derste verildi):** `current`'ın eski `next`'i (sonraki düğüm) kaybedilir ve listenin geri kalanına erişilemez. Doğru sıra: önce `tmp->next = current->next;`, sonra `current->next = tmp;`.

Soru 2 (derste sözlü, 01:15:14): `insert_top` fonksiyonunda yeni düğüm oluşturulup verisi yazıldıktan ve `new_node->next = head` yapıldıktan sonra iş bitmiş midir?

> **Çözüm (derste verildi):** Hayır. `head = new_node` ile `head` güncellenmeli, ardından `return head` ile yeni baş çağırana döndürülmelidir.

Soru 3 (derste sözlü, 01:21:42): Silmede yalnız `current->next = current->next->next;` yazmak yeterli midir?

> **Çözüm (derste verildi):** Listeden çıkarma açısından yeterlidir, ama silinen düğüm bellekte erişilemez hâlde yer kaplamaya devam eder. Doğrusu: `b = a->next; a->next = b->next; free(b);`.

Soru 4 (derste sözlü, 02:12:26): $n = 9$ kişilik bir çemberde her 5. kişi elenirse son 4 kişi kimlerdir? Tek kişi kalana kadar devam edilirse sona kim kalır?

> **Çözüm (derste verildi):** Elenme sırası 5, 1, 7, 4, 3, 6, 9, 2'dir. Son 4 kişi 2, 6, 8, 9; sona kalan tek kişi 8'dir (Bölüm 13.4).

Soru 5 (derste sözlü, 02:11:42): Josephus, 1'den 41'e kadar numaralı çemberde her 3. kişi öldürülürken sona kalacak iki numarayı nasıl buldu? Bu iki numara hangileridir?

> **Çözüm (derste verilmedi):** Derste yalnız soru soruldu; $n = 41$, $m = 3$ için cevap verilmedi. Bölüm 13.5'teki dairesel liste koduyla ($N = 41$, $M = 3$, $M - 1 = 2$ kişi kalınca durarak) ya da python simülasyonuyla bulunur: sona kalan iki numara **16 ve 31**'dir. Tek kişi kalana kadar devam edilseydi son kalan 31 olurdu.

Dersin Toparlaması

Bu derste diziler hızlıca hatırlatıldıktan sonra ilk temel veri yapısı olan bağlantılı listeye geçildi. Dizide indisle $O(1)$ erişim vardır, ama araya ya da başa ekleme ve silme kaydırma gerektirdiği için $O(n)$'dir. Bağlantılı listede ise düğümler veri ve `next` işaretçisi taşır; listeye yalnız `head` üzerinden erişilir ve son düğüm `NULL`'u gösterir. Yer bulunduktan sonra ekleme ve silme sabit sayıda işaretçi güncellemesiyle yapılır. Bu güncellemelerin sırası, listenin geri kalanını kaybetmemek için kritiktir. Silinen düğüm mutlaka `free` edilmelidir. Akış diyagramlarında liste, bir veri dizisi (`A`) ve bir link dizisi (`L`) ile gösterildi; sıralı listeye ekleme ve silme bu gösterimle adım adım izlendi.

Ardından bağlantılı listenin türleri görüldü. Insertion sort, düğümleri bir listeden koparıp öbürüne takarak kaydırmasız yapıldı. Çift yönlü listede araya ekleme dört, silme iki işaretçi güncellemesi ve bir `free` ile yapıldı. Dairesel listede son düğüm başa bağlandı ve Josephus problemi tek bir döngüyle, `if` kullanmadan çözüldü. Son olarak dizi ile bağlantılı liste karşılaştırıldı: Rastgele erişim ve sıralı veride ikili arama gerekiyorsa dizi, sık araya ekleme/silme ve sıralı erişim varsa bağlantılı liste uygundur. Hoca bir sonraki hafta yığın (stack) veri yapısına geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.